

Bloc 4

Maintenir l'application logicielle en condition operationnelle

Dossier de preuves - RoundLab

Application web d'analyse de demos Counter-Strike 2

Candidat	A completer
Branche	codex/bloc-4-maintenance-operationnelle
Version candidate	0.1.41
Date	3 aout 2026
Application	https://lakav.github.io/RoundLab/

Principe du dossier : chaque affirmation est reliee a une preuve technique. Les automatisations non encore fusionnees sont marquees comme pretes sur la branche. Aucun echange client ou support n'est invente.

1. Synthese executive

RoundLab est un site statique Next.js deploye sur GitHub Pages. Le parseur Rust compile en WebAssembly traite les fichiers GOTV localement dans le navigateur. La maintenance porte donc sur le frontend, le parseur, les donnees statiques, les dependances, la CI et la disponibilite des artefacts publies.

Le depot possedait deja une CI exigeante, des audits de securite, des tests de non-regression et plusieurs correctifs reels. La branche Bloc 4 ajoute les pieces manquantes : Dependabot, un manifeste de deploiement, une sonde planifiee, des alertes par issue, un formulaire de bug, un changelog et des fiches de preuve.

C4.1.1	Prete sur la branche	Dependabot + audits npm/Cargo	Premieres PR periodiques
C4.1.2	Prete sur la branche	Sonde, manifeste, WASM, incident	Runs distants apres fusion
C4.2.1	Outillage complet	Feedback, issue form, registre	Issue utilisateur reelle
C4.2.2	Demontree	Correctifs, tests, CI, deploiement	Livraison de 0.1.41
C4.3.1	Demontree	Plan priorise et P1 realisees	Mesures sur une semaine
C4.3.2	Prete sur la branche	Version 0.1.41 + CHANGELOG	Tag apres validation
C4.3.3	Preuve humaine absente	Processus et fiche prepares	Echange reel avec un tiers

Limite majeure : au 3 aout 2026, le depot GitHub ne contient aucune issue et aucune revue externe. Le critere C4.3.3 ne peut pas etre declare acquis sans interlocuteur reel.

2. Architecture maintenue

web/	Interface, replay et analyses TypeScript	regression fonctionnelle ou performance
parser/	Extraction Rust des faits GOTV	donnees incorrectes ou import bloque
WASM	Execution locale du parseur	artefact absent, corrompu ou trop volumineux
IndexedDB	Stockage local des matchs	migration ou compatibilite des anciens imports
GitHub Actions	CI, audits, deploiement et sonde	controle absent ou faux positif
GitHub Pages	Hebergement statique	indisponibilite ou ressources manquantes

3. C4.1.1 - Mise a jour des dependances

Etat existant

- Dependances web verrouillees par pnpm et dependances Rust verrouillees par Cargo.
- Node 24, pnpm 11.9.0, Rust 1.95.0 et wasm-bindgen 0.2.126 sont fixes dans la chaine.
- pnpm audit --audit-level high et cargo audit bloquent la CI.
- Le commit 815fd82 du 30 juillet 2026 documente une correction de dependances vulnerables.
- L'audit du 3 aout corrige RUSTSEC-2026-0190 et RUSTSEC-2026-0186 dans Cargo.lock.

Automatisation ajoutee

Le fichier .github/dependabot.yml programme chaque lundi trois controles separes : npm dans web/, Cargo dans parser/ et GitHub Actions a la racine. Cinq PR simultanees au maximum sont autorisees par ecosysteme. Aucune fusion automatique n'est activee.

Ordre et type de mise a jour

1	Securite critique exploitable	analyse immediate, mesure ou correctif sous 72 h
2	Rupture de compatibilite	branche dediee et plan de retour
3	Correctif ou mineure	lot mensuel apres CI complete
4	Majeure	une migration isolee avec notes de version

Sequence de validation

lockfile fige -> audit -> tests -> lint et typage -> build -> accessibilite -> audits applicatifs -> revue -> fusion -> controle post-deploiement

Preuve et limite

La configuration prouve que le processus est implemente. La regularite ne sera prouvee qu'apres plusieurs PR Dependabot datees. Le commit 815fd82 ne conserve ni CVE ni sortie avant/apres : aucun identifiant de vulnerabilite n'est donc affirme dans ce dossier.

4. C4.1.2 - Supervision et alertes

Topologie

Avant fusion	audit, tests, typage, build, accessibilite	echec de PR
Avant deploiement	budget JS/WASM, export statique, integrite	deploiement bloque
Toutes les 6 heures	accueil, feedback, health.json, WASM	run et rapport
Incident	echec ou seuil critique	issue ouverte ou commentee
Retablissement	sonde a nouveau valide	preuve ajoutee et issue fermee

Sondes et finalite

- Accueil : code HTTP et presence du nom RoundLab.
- Feedback : code HTTP et presence du formulaire de signalement.
- Manifeste : schema, version, commit, date, routes et identite du WASM.
- WASM : taille, entete WebAssembly et SHA-256 exact.
- Performance : avertissement au-dela de 2 s, echec au-dela de 5 s.
- Conservation : rapports JSON et Markdown gardes 90 jours.

Baseline du 3 aout 2026

/RoundLab/	HTTP 200	site public disponible
/RoundLab/feedback/	HTTP 200	canal de signalement disponible
/RoundLab/health.json	HTTP 404	normal avant deploiement de 0.1.41

La sonde est testee localement mais n'est pas encore active en production. Le premier run distant reussi apres fusion sera la preuve d'activation.

Confidentialite

Aucun fichier .dem ni log navigateur n'est envoye automatiquement. La supervision controle uniquement des ressources publiques. Le diagnostic utilisateur reste volontaire et doit etre verifie avant publication.

5. C4.2.1 - Consignation des anomalies

La page /feedback prepare une issue GitHub. Le nouveau formulaire natif exige categorie, impact, version, navigateur, resultat observe, reproduction, resultat attendu et confirmation de confidentialite. La console Ctrl/Cmd + Shift + D permet de copier les diagnostics locaux.

Cycle de traitement

1. Collecter	formulaire ou issue	signalement date
2. Qualifier	impact, version, environnement	priorite P0 a P3
3. Reproduire	scenario minimal	resultat observe et attendu
4. Diagnostiquer	cause et hypotheses rejetees	commentaire ou fiche
5. Corriger	branche dediee	diff et commit
6. Tester	non-regression	resultat CI
7. Livrer	fusion et deploiement	commit et run
8. Clore	controle et retour utilisateur	confirmation finale

Registre initial

B4-001	croissance memoire du replay	P1 estime	19a13c8	tests replay/store/Pixi
B4-002	import de grande demo	P1 estime	c995760	seuil, Worker et tests
B4-003	dependances web vulnerables	indetermine	815fd82	lockfile et audit CI
B4-004	avis RustSec anyhow/memmap2	P2 estime	Cargo.lock	audit final sans avis

Les severites B4-001 et B4-002 sont reconstruites a partir de l'impact. B4-003 reste indeterminee faute d'avis original. Ces limites sont conservees dans le registre.

6. C4.2.2 - Correctif et deployment continu

Cas principal : import d'une grande demo

Le commit c995760 du 22 juillet 2026 corrige un depassement de memoire. Le decodeur Zstd et le parseur WASM pouvaient conserver simultanement de gros tas, tandis que l'echantillonnage complet pouvait atteindre la limite memoire 32 bits de WebAssembly.

- Decompression transferee dans un Worker court puis termine avant le parseur.
- Buffer transfere sans copie inutile.
- Strategie sure a partir de 384 Mio.
- Mode precis refuse au-dessus du seuil ; mode rapide maintenu.
- Erreurs memoire converties en message actionnable.
- Tests de seuil, de strategie et de message d'erreur.

Deuxieme cas : croissance memoire du replay

Le commit 19a13c8 borne les payloads de rounds conserves, limite la file d'objets PIXIJS detaches et detruit les ressources graphiques. Les tests verifient le voisinage des rounds, la vue condensee et la borne de destruction.

Chaine de livraison prouvee

PR #2	CI frontend et Rust reussie
PR #3	deployment puis rollback consignes
PR #5	restauration et smoke test consignes
Run 30799703961	CI main du 3 aout reussie
Run 30799877146	deployment correspondant reussi
CI echouees des 27 et 30 juillet	deployements automatiques ignores

Ces preuves demontrent le fonctionnement du flux CI/deployment et du rollback. La version 0.1.41 devra produire ses propres liens avant soutenance.

7. C4.3.1 - Axes d'amelioration

Realise	sonde planifiee	detection proactive	0,5 a 1 j	succes et delai
Realise	Dependabot	reduction exposition	0,5 j	age des dependances
Realise	changelog	versions auditablees	15 min/livraison	versions documentees
Realise	issue form	triage plus rapide	0,5 j	tickets reproductibles
A faire	smoke avec vraie demo	validation metier	1 a 2 j	parcours reussi
A etudier	erreurs opt-in	visibilite navigateur	2 a 4 j	erreurs par version

Analyse cout/benefice

Les priorites retenues sont peu couteuses et adaptees a un site statique en beta. Une plateforme complete d'observabilite serait disproportionnee. Une collecte automatique de demos serait mauvaise sans consentement, minimisation et duree de conservation explicites.

8. C4.3.2 - Journal des versions

Le fichier CHANGELOG.md devient la source de changement livrable. La version candidate 0.1.41 est declaree dans web/package.json. Son commit, sa CI, son deploiement et son premier run de supervision restent volontairement 'a completer' jusqu'a leur existence reelle.

0.1.41	candidate Bloc 4	changelog present ; livraison a prouver
0.1.40	candidate du 21 juillet	commit de30fd8 et PR #2/#3/#5 ; aucun tag final
anciennes v0.1.x	historique nettoye	tags et releases supprimees via PR #11

9. C4.3.3 - Collaboration support et client

Outillage disponible

- page de signalement integree et formulaire GitHub structure ;
- console de diagnostic copiable ;
- messages d'erreur actionnables ;
- fiche de collaboration avec roles, chronologie et confirmation ;
- checklist de confidentialite avant publication.

Preuve manquante

Aucune issue GitHub, revue externe ou validation d'un tiers n'existe au 3 aout 2026. L'outillage ne remplace pas une collaboration reelle.

Preuve minimale a obtenir

Signalement	demande datee d'un utilisateur, client ou support
Clarification	questions, contexte et resultat attendu
Diagnostic	explication comprehensible et contributions des parties
Livraison	correctif, tests, commit et deploiement
Validation	confirmation explicite du tiers

Si aucun tiers n'est disponible avant le jury, la bonne presentation consiste a reconnaitre que ce critere n'est pas acquis. Une fausse conversation serait facile a contester et affaiblirait tout le dossier.

10. Plan de preuves avant soutenance

Fusionner la branche apres CI	a faire
Conserver le run CI 0.1.41	a faire
Conserver le deploiement et health.json	a faire
Executer manuellement la sonde	a faire
Conserver plusieurs runs planifies	a faire
Traiter une premiere PR Dependabot	a faire
Documenter un vrai echange utilisateur	preuve humaine obligatoire
Creer un tag seulement apres validation	a faire

11. Index des preuves

CI	.github/workflows/ci.yml et _checks.yml
Deploiement	.github/workflows/deploy-pages.yml
Supervision	.github/workflows/monitor-production.yml
Sonde	scripts/monitor-production.py
Manifeste	scripts/write-deployment-manifest.py
Test de sonde	scripts/tests/test_monitor_production.py
Dependances	.github/dependabot.yml
Anomalies	.github/ISSUE_TEMPLATE/bug-report.yml
Registre	docs/bloc-4/registre-anomalies.md
Processus	docs/bloc-4/processus-maintenance.md
Versions	CHANGELOG.md
Collaboration	docs/bloc-4/fiche-collaboration-support.md
Preuves datees	docs/bloc-4/rapport-preuves-2026-08-03.md

Sources officielles utilisees pour la configuration GitHub

- docs.github.com - About the dependabot.yml file
- docs.github.com - Keeping your actions up to date with Dependabot
- docs.github.com - Scheduling issue creation
- docs.github.com - Syntax for issue forms
- docs.github.com - Workflow commands and job summaries

Conclusion

La branche couvre techniquement les attendus de maintenance : veille, controles, sonde, alertes, anomalies, correctifs, recommandations et versions. Les preuves historiques de CI, deploiement et rollback sont disponibles. Deux validations restent necessairement futures : l'activation distante de 0.1.41 et une collaboration reelle avec un tiers.